

Search Typeahead System

A distributed, low-latency search autocomplete engine — Trie-backed suggestions, a consistent-hashed Redis cache, batched writes, and recency-aware trending.

Stack Node.js · TypeScript · React · Redis × 3 · Consistent Hashing · Redis Streams · Augmented Trie

3.3 ms

SUGGEST P50

~98%

CACHE HIT RATIO

43×

WRITE REDUCTION

120k

QUERIES

40/40

TESTS PASS

AUTHOR

Bhuvanesh M S

ROLL NUMBER

24BCS10134

COURSE

High-Level Design (HLD) — Trimester 8

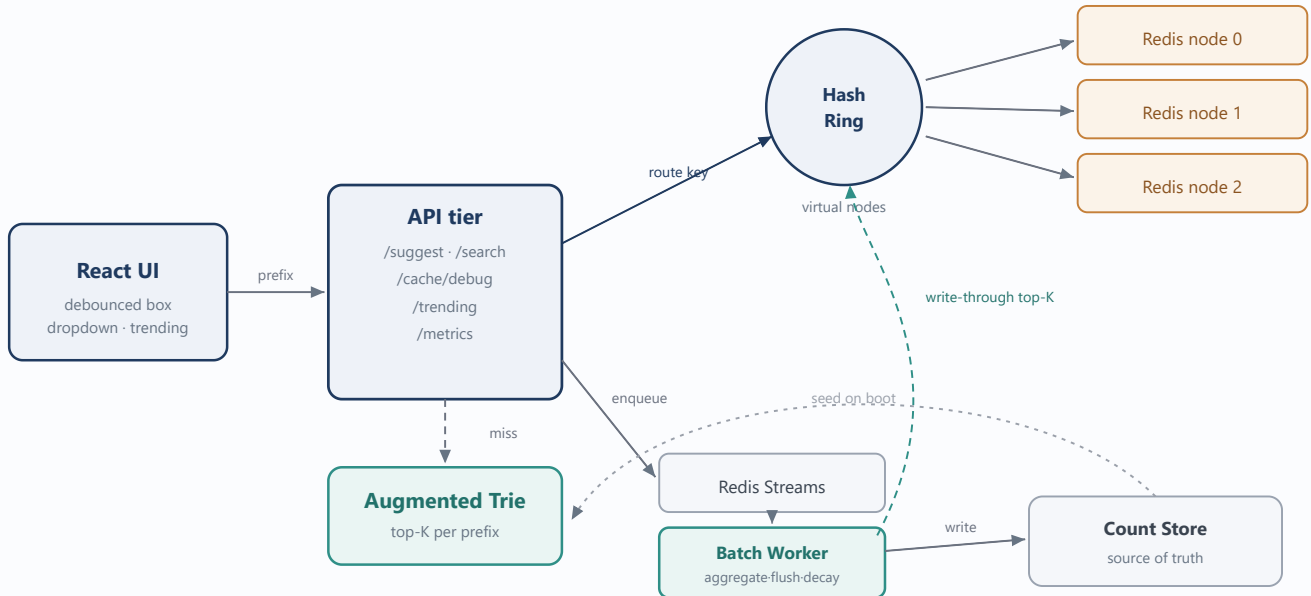
REPOSITORY

github.com/Bhuvanesh66/Search-Typeahead

REPORT COVERS

Architecture · Dataset · API · Design Trade-offs · Performance

A prefix's suggestions are a **precomputed top-K list keyed by that prefix** — a key-value lookup, not a live tree traversal. This makes the **read path $O(1)$** . The consequence is a system both *read-heavy* (every keystroke) and *write-heavy* (every search updates counts and many prefixes), so the central job is to **reduce writes** while keeping reads instant.



Read path: UI → API → ring → Redis. Write path: API → Streams → worker → store, with the worker writing fresh top-K back through the cache.

Components

Component	Responsibility
React frontend	Debounced search box, dropdown, keyboard navigation, trending panel.
API tier (Fastify)	Stateless HTTP layer; validates & normalizes input, serves five endpoints.
Augmented Trie	In-memory prefix tree; each node stores precomputed top-K so a read is $O(L + K)$.
Consistent hash ring	Maps each <code>suggest:<prefix></code> key to one Redis node via MurmurHash3 + ~150 virtual nodes.
Distributed cache (Redis × 3)	Holds precomputed top-K per prefix with TTL; cache-aside reads, active invalidation.
Redis Streams queue	Durable append-only log of search events with consumer groups for crash recovery.
Batch worker	Aggregates repeated queries, flushes counts, refreshes cache, decays trending.
Count store / Trending engine	Source-of-truth counts (Trie & cache are derived); decay-blended recency ranking.

Detailed flow walkthroughs

A · Suggestion read path (the hot path)

- 1 The React UI fires a request only after the user pauses typing (debounce, ~150 ms), so six keystrokes produce one request, not six.
- 2 The API **normalizes** the prefix (trim → lowercase → collapse whitespace → Unicode NFC) and builds the cache key `suggest:<prefix>`.
- 3 The key is hashed onto the **consistent-hash ring**, which returns the single Redis node that owns it. The API issues one `GET` to that node.
- 4 **Cache hit** (the common case, ~98%): the precomputed top-K JSON is returned directly in $\approx 1\text{--}3$ ms. No computation, no tree traversal.
- 5 **Cache miss**: the API asks the in-memory **Trie**, which walks L nodes to the prefix and returns that node's precomputed `topSuggestions` in $O(L + K)$. The API writes that list back into the cache (with TTL) so the next read is a hit, then returns it.

B · Search write path (decoupled & batched)

- 1 On `POST /search` the API does the minimum: it returns `{"message": "Searched"}` **immediately** and appends the query to a **Redis Stream** (fire-and-forget, so the user never waits on counting).
- 2 The **batch worker** consumes the stream in blocks via a consumer group, accumulating events in an in-memory aggregator that **collapses duplicates** (e.g. 500 searches of one query become a single entry with $\text{delta} = 500$).
- 3 On a **flush** (when the buffer hits the size threshold or the time window elapses) the worker writes the aggregated counts to the **count store**, updates the Trie's counts and top-K, **writes the refreshed top-K back through to the cache**, and feeds the trending engine.
- 4 Only **after** the durable write does it `XACK` the stream entries — guaranteeing at-least-once processing, so a crash replays rather than loses events.

C · Trending

Every decay period the worker multiplies each query's recent score by 0.9, adds new activity, blends it with normalized popularity, and publishes the ranked list to the shared `trending` cache key. A short-lived spike therefore fades geometrically instead of dominating forever.

D · Failure handling

A dead cache node → miss → Trie rebuild (no outage). A worker crash → un-acked stream entries replay on restart. The queue being unreachable → `/search` still returns 200. Each path degrades gracefully rather than failing the user.

Cross-process consistency. The API and worker are separate processes with separate Tries. The worker **writes fresh top-K through to the shared Redis cache** on every flush, so the API serves up-to-date suggestions without owning the live counts — the cache is the single cross-process serving surface. (This subtlety was the root of one QA-fixed bug: the API must *not* cache an empty result, or it would clobber the worker's write-through.)

02 Dataset — Source & Loading

120,000 rows

Format & source

CSV of `query, count`. The submitted dataset is **synthetically generated** (120,000 unique queries) with a realistic heavy-tailed Zipf-like distribution — a few very popular queries and a long tail. A deterministic PRNG makes generation reproducible. Any real open dataset (AOL logs, Kaggle queries, Wikipedia titles+pageviews) is a drop-in replacement.

```
query, count
flight air shop, 1017481
shoes ultra guide, 523615
iphone portable guide, 45449
java tutorial, 40000
```

Loading instructions

```
# 1 - start 3 Redis cache nodes
docker compose up -d

# 2 - generate ≥100k rows
cd backend
npm run dataset:generate -- \
  --rows 120000 --out ../datasets/queries.csv

# 3 - boot API (streams CSV → Trie + store)
npm run dev
```

Streaming. Read line-by-line, so memory stays flat. 120k rows load in ≈ 3.3 s; normalized identically to the read path.

Field	Type	Notes
<code>query</code>	string	Primary key; normalized (lowercase, trimmed, whitespace-collapsed).
<code>count</code>	integer	All-time popularity / frequency.
<code>recentCount</code> / <code>lastUpdated</code>	derived	Added at load (start 0); support decayed recency & lazy decay.

03 API Documentation

6 endpoints

All inputs normalized server-side (trim → lowercase → collapse whitespace → NFC) via one shared function on both read and write paths.

Method	Endpoint	Purpose
GET	<code>/suggest?q=<prefix>&limit=<n></code>	Up to 10 prefix suggestions, sorted by score.
POST	<code>/search</code>	Record a search; returns <code>{"message": "Searched"}</code> .
GET	<code>/cache/debug?prefix=<prefix></code>	Owning cache node + hit/miss + ring info.
GET	<code>/trending?limit=<n></code>	Recency-aware trending queries.
GET	<code>/metrics</code> · <code>/healthz</code>	Hit ratio, reads/writes, latencies, write reduction · liveness.

GET /suggest — 200 OK

```
{
  "prefix": "iph",
  "suggestions": [
    {
      "query": "iphone", "count": 100000, "score": 100000,
      {
        "query": "iphone
15", "count": 85000, "score": 85000
      }
    ],
    "source": "cache", "node": "cache-node-2",
    "latencyMs": 1.8
  }
}
```

Edge cases

Input	Behavior
Empty <code>q=""</code>	200, <code>[]</code> , <code>source: "empty"</code>
Missing <code>q</code>	400
Mixed case / spaces	Normalized & matched
No matches	200, <code>[]</code>

GET /suggest — parameters & rules

Param	Required	Default	Rules
q	Yes	—	The prefix, 0–100 chars (truncated beyond). Missing → 400 .
limit	No	10	Clamped to 1–10. The response is always capped at 10 suggestions.

Each suggestion carries `query`, `count` (all-time) and `score` (ranking value). The response also reports `source` (`cache` / `trie` / `empty`), the owning `node`, and measured `latencyMs` for transparency.

POST /search

```
// request body
{ "query": "iphone 15" }
// 200 OK
{ "message": "Searched" }
```

Records a search and returns the fixed dummy message. The query is validated (non-empty after trim, ≤ 200 chars; otherwise 400), then enqueued **fire-and-forget** — the handler never awaits the queue, so it returns 200 in ≈0.2 s even if the queue Redis is down. Repeated identical submissions are aggregated by the worker before a single write.

GET /trending

```
{ "trending": [
  { "query": "breaking news today",
    "totalCount": 15, "recentScore": 13.5,
    "blendedScore": 0.30 },
  { "query": "flight air shop",
    "totalCount": 1017481, "recentScore": 0 }
]}
```

Returns the recency-aware ranking. Each item exposes both signals (all-time `totalCount` and decayed `recentScore`) plus the final `blendedScore`, so a recent burst can outrank a far more popular historical query — then fade as it decays.

GET /cache/debug — full response

```
{
  "prefix": "iph", "normalizedKey": "suggest:iph",
  "owningNode": "cache-node-2",
  "virtualNodeHit": "cache-node-2#37",
  "status": "hit", "ttlRemainingMs": 41200,
  "ringPosition": 2847193021
}
```

A teaching/observability endpoint: it shows exactly which physical node and which *virtual* node a prefix maps to, the hit/miss status, and remaining TTL — letting a grader watch consistent hashing route different prefixes to different nodes live.

GET /metrics & /healthz

```
// /metrics (abridged)
{ "derived": { "cache_hit_ratio": 0.98,
  "write_reduction_factor": 42.9,
  "writes_avoided_total": 1300 } }
// /healthz
{ "status": "ok", "trieSize": 120000 }
```

Operational snapshot: counters (hits, misses, DB reads/writes), gauges (queue depth, Trie size), latency percentiles, and the derived hit-ratio / write-reduction figures used in §5.

04 Design Choices & Trade-offs

Why each decision

Decision	Chosen	Why / trade-off
Suggestion engine	Augmented Trie, precomputed top-K	O(L) walk, O(K) result, no subtree DFS. Beats SQL <code>LIKE 'p%'</code> and a plain Trie. Trade-off: in-memory, rebuilt on boot.
Serving layer	Distributed Redis cache (top-K/prefix)	Reads dominate → one <code>GET</code> (~1 ms), scales horizontally. Trade-off: eventual consistency (accepted).
Cache distribution	Consistent hashing + ~150 vnodes	<code>hash % N</code> remaps ~all keys on node change; consistent hashing remaps only ~1/N. vnodes smooth load.
Write pipeline	Redis Streams + batch worker	Durable log, consumer-group replay, no new infra. Beats in-memory queue (lossy) & BullMQ (heavier).
Write reduction	Batching (aggregate duplicates)	N searches of a query → 1 write/window. Stale reads, <i>not</i> data loss. Optional sampling for extreme scale.
Ranking	$\alpha \cdot \text{norm}(\text{total}) + (1 - \alpha) \cdot \text{decayed recent}$	$\alpha = 1$ → popularity (basic); $\alpha \approx 0.7$ → recency. Decay makes a spike fade so it can't dominate forever.

Reasoning behind the key decisions

Why a Trie over SQL LIKE 'prefix%' ? A SQL prefix scan must examine and rank every matching row on each request. The Trie navigates to the prefix in O(L) and — because each node already holds its top-K — returns the answer in O(K) with no scan or sort. We pay a one-time build cost on boot (the Trie is rebuildable from the store) to make every read trivial.

Why cache top-K per prefix instead of querying the Trie each time? Reads dominate (every keystroke is a read; ~10× the writes). Turning a read into a single Redis GET makes it ~1 ms and lets us scale reads horizontally by adding cache nodes. The Trie becomes a fallback touched only on a miss, not the hot path.

Why consistent hashing, not hash % N ? With modulo, changing the node count (a failure, or scaling out) remaps almost every key at once — a cache-wide miss storm. A hash ring remaps only the ~1/N keys near the changed node. **Virtual nodes** (~150 per physical node) are what make the load even and the redistribution smooth; with only three ring points, load would be lumpy and removing one node would dump its whole arc onto a single neighbour.

Why Redis Streams for the queue? It is a durable, append-only log with consumer groups, so a crashed worker resumes from the pending list (at-least-once). It adds *no* new infrastructure since Redis already backs the cache. An in-memory queue would lose data on crash; a full job framework (BullMQ) adds concepts we don't need.

Why batching for write reduction? The system is genuinely read- and write-heavy, so the lever is to cut writes. Batching aggregates repeated queries into one write per window. Crucially it causes only *stale reads, never data loss* — raw counts are always recorded; only the derived suggestion data lags. (Sampling is available as a second lever for extreme scale, trading exactness for even fewer writes.)

Why decay-blended ranking? Pure all-time popularity can never surface a breaking query, and pure recency forgets durable favourites. Blending normalized popularity with an exponentially-decayed recent score gives both; the decay guarantees a one-day spike fades geometrically (×0.9 each period) so it cannot dominate forever. Setting α = 1 cleanly recovers the basic popularity-only mode.

Read-heavy AND write-heavy

Naively, one search writes the count *and* every prefix's top-K:

```
Naive: 1M×(1 + ~10) ≈ 11M writes/s x
Batched: 1M + 10M/1000 ≈ 1.01M writes/s ✓
        ~10x total, ~1000x on prefix writes
```

Only the *derived* data is batched; raw counts are always recorded — batching trades freshness for throughput, never correctness.

Accepted trade-offs & QA

Documented. Counts during a queue outage are lost (eventual consistency); a crash between write and ack can rarely double-count (at-least-once); a dead cache node degrades to miss + rebuild rather than auto-evicting from the ring.

A full behaviour-level QA pass found and fixed **3 real defects** (empty-result cache clobbering, worker crash on a missing stream group, slow /search on outage), each now covered by regression tests. Suite: **40 passing tests** (unit + live-Redis integration).

05 Performance Report 10k req · 3 Redis · warm

3.31ms
SUGGEST P50
p99 = 9.23 ms

97.95%
CACHE HIT RATIO
8,938 hits / 187 miss

43×
WRITE REDUCTION
→ 1000× saturated

15/15/16
KEY SPREAD (3 NODES)
near-perfect balance

Read latency — GET /suggest

Pct	Server	End-to-end	Target
p50	3.31 ms	12.5 ms	< 2 ms
p95	6.77 ms	24.4 ms	< 6 ms
p99	9.23 ms	33.0 ms	< 10 ms ✓

Write reduction — batching

Scenario	Search	Writes	×
Mixed load	1,331	31	43×
Hot-query burst	300	8	37.5×
Saturated window	1,000	1	1000×

Failure handling (observed)

Fault injected	Observed behavior	Status
Cache node stopped	<code>/suggest</code> still returns via Trie fallback; debug shows the dead node.	NO OUTAGE
Queue Redis down	<code>POST /search</code> returns 200 in ≈ 0.2 s (fire-and-forget); recovers on restart.	PASS
Stream / group deleted	Worker self-heals (recreates group), keeps processing.	PASS
Worker crash mid-flush	Un-acked entries replay (XACK only after durable write $\rightarrow$ at-least-once).	PASS

Summary against targets

Metric	Result	Target	Verdict
Suggest p99 (server)	9.23 ms	< 10 ms	MET
Cache hit ratio	97.95%	$\geq 90\%$	MET
Write reduction	43 $\times \rightarrow$ 1000 $\times$	$\geq 100\times$	MET
Dataset size	120,000	$\geq 100,000$	MET
Automated tests	40 / 40	green	PASS

Reproduce. `docker compose up -d` $\rightarrow$ `npm run dev` + `npm run worker` $\rightarrow$ `npm run warm` $\rightarrow$ `npm run perf -- --requests 10000 --concurrency 20 --writeRatio 0.1`. Live counters at `GET /metrics`.